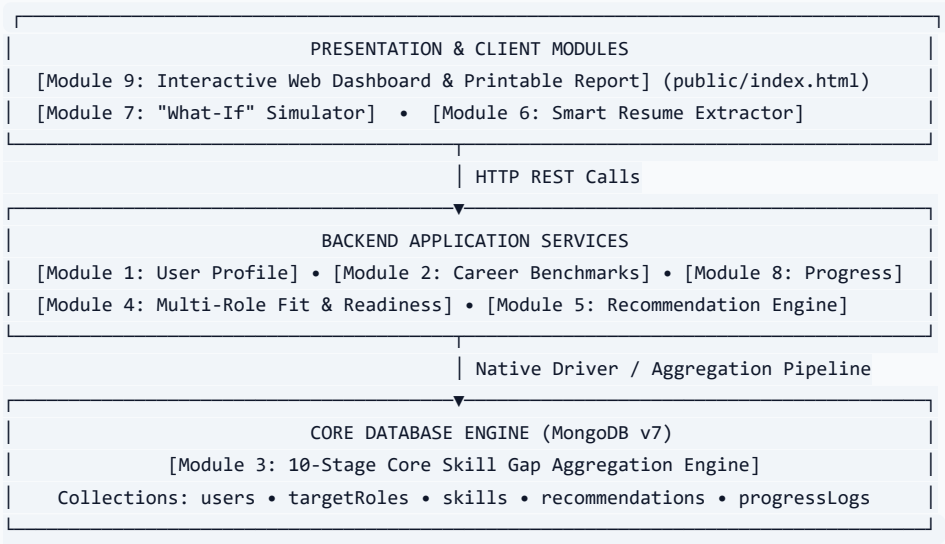
Personalized Skill Gap Tracking & Recommendation System Using MongoDB

In-Depth Architectural & Module-by-Module Technical Specification

Project: Personalized Skill Gap Tracking & Recommendation System Using MongoDB
Degree: B. Tech in Computer Science and Engineering
School: School of Computer Science and Engineering (SCOPE), VIT
Authors: - **Tharakesha M** (24BCE2538) - **Mathu Metha S** (24BCE2805) - **Charulatha Venkatraman** (24BCE0977) - **Jai Jagannayagam** (24BCE2563)

Overall System Architecture



Module 1: User Profile & Skill Management Module

1. Purpose & Responsibilities

Manages learner identities, registration details, career aspirations, and current technical competencies. It acts as the primary data ingestion point for candidate self-assessments.

- **Primary Files**: `src/routes/users.ts`
- **Associated Collection**: `users`
- **REST Endpoints**:
- `POST /api/users` — Registers a candidate.
- `GET /api/users` — Lists all registered candidates.
- `GET /api/users/:id` — Retrieves profile details by ObjectId.
- `PUT /api/users/:id` — Updates skills array or switches target career role.
- `DELETE /api/users/:id` — Removes candidate and cleans up records.

2. MongoDB Design Pattern: Hybrid Embedding

- **Embedded Array**: The `skills` list is stored directly inside the user document: `json "skills": [ { "name": "React", "level": 3 }, { "name": "Python", "level": 4 } ]` *Why*: Candidate skills are 1:N bounded (a developer typically has 5–30 skills). Storing them as an embedded array guarantees **single-seek atomic reads**, eliminating the need for expensive relational JOIN tables (`UserSkills`).
- **Normalized Reference**: The `targetRoleId` stores a foreign `ObjectId` referencing the `targetRoles` collection: `json "targetRoleId": ObjectId("6acf82d03158599022ca0321")` *Why*: Job requirements are shared benchmarks. Referencing avoids duplicating job definitions across thousands of users and allows global updates to career standards.

3. Workflow & Internal Logic

1. When a user submits skills, the server validates proficiency constraints ($1 \le \text{level} \le 5$).
2. Uses MongoDB's atomic operator **`$set`** to update the document without replacing unchanged fields: `typescript await db.collection("users").updateOne( { _id: new ObjectId(req.params.id) }, { $set: updateFields } );`

Module 2: Career Benchmark & Role Taxonomy Module

1. Purpose & Responsibilities

Maintains an industry-calibrated repository of career roles across **all 7 domains of Computer Science** (Software Dev, AI/Data, Cloud/DevOps, Cybersecurity, Systems/Hardware, Database Architecture, Management/QA).

- **Primary Files**: `src/routes/targetRoles.ts` , `src/data/roles.json`
- **Associated Collection**: `targetRoles`
- **REST Endpoints**:
- `GET /api/roles` — Returns all 22 CS roles grouped by domain.
- `GET /api/roles/:id` — Returns single role benchmark specifications.
- `POST /api/roles` — Creates custom industry benchmark profiles.

2. Schema Structure

Each career role is a flexible document containing an array of requirement sub-documents:

```
{
  "_id": ObjectId("6acf82d03158599022ca0321"),
  "title": "Backend Engineer",
  "domain": "Software Development",
  "description": "Architects high-throughput microservices and database persistence layers.",
  "requirements": [
    { "name": "Node.js", "requiredLevel": 4, "category": "Backend", "weight": 3 },
    { "name": "MongoDB", "requiredLevel": 4, "category": "Database", "weight": 3 },
    { "name": "System Design", "requiredLevel": 4, "category": "Architecture", "weight": 3 },
    { "name": "REST API Design", "requiredLevel": 4, "category": "Backend", "weight": 2 },
    { "name": "Docker", "requiredLevel": 3, "category": "DevOps", "weight": 2 }
  ]
}
```

3. Key Technical Highlight

- **Weights ($W \in [1, 3]$)**: Differentiates **Core Must-Have Skills** (Weight 3) from **Secondary Skills** (Weight 2) and **Supporting Tools** (Weight 1).
- **Schema Heterogeneity**: Roles have varying numbers of requirements (e.g. *Full Stack* has 6 requirements; *AI Engineer* has 6; *Security Engineer* has 4). MongoDB handles this variance natively without null columns.

Module 3: Core Skill Gap Recognition & Aggregation Engine

1. Purpose & Responsibilities

The computational heart of the system. It compares the candidate's current proficiencies against target benchmarks, calculates raw gaps, evaluates weighted urgency, and assigns status categories.

- **Primary Files**: `src/routes/skillGap.ts`
- **REST Endpoint**: `GET /api/skill-gap/:userId`
- **Execution Environment**: **100% inside MongoDB server memory in C++**.

2. The 10-Stage Pipeline Deep Dive

Stage	Operation	Code Implementation	Purpose
1	`$match`	`{$match: { _id: userId }}`	Isolates the single active candidate document.
2	`$lookup`	`{$lookup: { from: "targetRoles", localField: "targetRoleId", foreignField: "_id", as: "targetRole" }}`	Left outer join bringing the benchmark requirements into the stream.
3	`$unwind`	`{$unwind: "$targetRole"}`	Flattens the joined single-element array into an object.
4	`$unwind`	`{$unwind: "$targetRole.requirements"}`	Deconstructs the array into individual stream documents (1 per skill).
5	`$addFields` + `$filter`	`$filter: { input: "$skills", as: "s", cond: {$eq: ["$$s.name", "$targetRole.requirements.name"]} }`	Finds the candidate's current proficiency for the specific benchmark requirement.
6	`$project` + `$ifNull`	`currentLevel: {$ifNull: ["$matchedSkill.level", 0]}`	Extracts the level. **Defaults to 0 if the skill was never recorded** (prevents null errors).
7	`$addFields` + `$max`	`gap: {$max: [{$subtract: ["$targetLevel", "$currentLevel"]}, 0]}`	Calculates $\text{Gap} = \max(L_{\text{target}} - L_{\text{current}}, 0)$. Clamps to 0 to prevent negative gaps if over-qualified.
8	`$addFields` + `$multiply`	`priorityScore: {$multiply: ["$gap", "$weight"]}`	Computes business urgency score. Categorizes status via `$cond` (`Missing`, `Needs Improvement`, `Satisfied`).
9	`$lookup`	`{$lookup: { from: "recommendations", localField: "skill", foreignField: "skillName", as: "learningResources" }}`	Dynamically joins matching courses for gaps where $\text{gap} > 0$.
10	`$sort`	`{$sort: { priorityScore: -1, gap: -1 }}`	Sorts the output with highest-urgency gaps first.

Module 4: Career Readiness & Multi-Role Fit Evaluation Module

1. Purpose & Responsibilities

Provides high-level quantitative analytics by calculating an overall **Career Readiness Score (0–100%)**, generating category-level rollups, and evaluating the student against all other 21 roles to identify alternative career pathways.

- **Primary Files**: `src/routes/skillGap.ts` (`calculateRoleFit` function)

2. Mathematical Formulations

a. Career Readiness Percentage (R)

$$R = \left(\frac{\sum_{i=1}^{N} \min(L_{\text{current}, i}, L_{\text{required}, i}) \times W_i}{\sum_{i=1}^{N} L_{\text{required}, i} \times W_i} \right) \times 100\%$$

- **Why $\min(L_{\text{current}}, L_{\text{required}})$ matters**: It prevents students from "gaming" the score. Having Level 5 in HTML cannot compensate for Level 0 in MongoDB for a Backend Engineer role. Points are capped at the required benchmark.

b. Multi-Role Fit Algorithm (Cross-Role Benchmarking)

The server queries all other roles:

```
const allRoles = await db.collection("targetRoles")
  .find({ _id: { $ne: targetRole._id } })
  .toArray();
```

It computes R across all other CS roles and returns the **Top 4 Alternative Career Matches** sorted by readiness percentage:

> *"You are **74% ready** for Backend Engineer, **61% ready** for Full Stack Developer, and **45% ready** for DevOps."*

c. Category Rollup Matrix

Aggregates total gaps by domain (Frontend, Backend, Database, Cloud) to reveal high-level weakness zones.

Module 5: Personalized Learning Recommendation System

1. Purpose & Responsibilities

Directly fulfills **Objective 4** of your lab report. Instead of presenting raw deficiencies, it maps each gap to verified, high-authority learning resources.

- **Primary Files**: `src/data/recommendations.json` , `src/seed.ts`
- **Associated Collection**: `recommendations`
- **Indexed On**: `{ skillName: 1 }` (enables $O(\log N)$ `$lookup` performance).

2. Resource Schema

```
{
  "_id": ObjectId("6acf82d03158599022ca0361"),
  "skillName": "MongoDB",
  "tier": "Beginner to Advanced",
  "title": "MongoDB Node.js Developer Path & Certification",
  "platform": "MongoDB University",
  "type": "Official Certification Path",
  "durationHours": 20,
  "url": "https://learn.mongodb.com"
}
```

3. Integration Mechanism

In Stage 9 of the Aggregation Pipeline, MongoDB joins matching resources using:

```
{
  $lookup: {
    from: "recommendations",
    localField: "skill",
    foreignField: "skillName",
    as: "learningResources"
  }
}
```

If a student has a gap in **React**, the pipeline automatically returns both the Udemy Comprehensive Course (~48h) and the React.dev Official Interactive Docs (~15h).

Module 6: Smart Resume & Bio Skill Extractor Module

1. Purpose & Responsibilities

Provides rapid, automated onboarding. Candidates can paste an unstructured bio or resume summary, and the system extracts standardized CS competencies.

- **Primary Files**: `src/routes/skills.ts` , `src/data/skills.json`
- **Associated Collection**: `skills` (39 Master CS competencies)
- **REST Endpoint**: `POST /api/skills/extract`

2. MongoDB Compound Text Index

```
await db.collection("skills").createIndex({
  name: "text",
  category: "text",
  description: "text"
});
```

3. Extraction Algorithm

```
for (const skill of allSkills) {
  const regex = new RegExp(`\\b${skill.name.replace(/[.*+?^${}()|[\]\\]/g, "\\$&")}\\b`, "i");
  const match = regex.test(normalizedText) ||
    (skillName === "react" && normalizedText.includes("reactjs")) ||
    (skillName === "mongodb" && normalizedText.includes("mongo"));
  if (match) detectedSkills.push({ name: skill.name, suggestedLevel: 3 });
}
```

- **User Value**: Candidate avoids manually clicking 30 dropdowns; extracted skills can be reviewed and applied to their profile with one click.

Module 7: Interactive "What-If" Skill Simulator Module

1. Purpose & Responsibilities

An interactive simulation engine for viva demonstrations and career planning. Candidates adjust hypothetical skill levels via UI sliders to preview score improvements without mutating stored database records.

- **Primary Files**: `src/routes/skillGap.ts` (`/simulate` endpoint)
- **REST Endpoint**: `POST /api/skill-gap/simulate`

2. Execution Payload & Logic

- **Request Payload**: `json { "targetRoleId": "6acf82d03158599022ca0321", "simulatedSkills": [ { "name": "React", "level": 4 }, { "name": "Node.js", "level": 4 }, { "name": "MongoDB", "level": 3 } ] }`
- **Processing**: The server reads the role benchmark from MongoDB, runs the mathematical formula in memory against the hypothetical skills, and returns the projected readiness score (R_{sim}) and updated gap metrics in under **5 milliseconds**.

Module 8: Historical Progress Tracking & Milestone Audit Module

1. Purpose & Responsibilities

Directly fulfills **Objective 5** of your lab report (tracking learning progress over time). Records an immutable audit log whenever a candidate completes coursework and levels up a skill.

- **Primary Files**: `src/routes/progress.ts`
- **Associated Collection**: `progressLogs`
- **REST Endpoints**:
- `POST /api/progress/:userId/level-up` — Upgrades a skill level and creates an audit snapshot.
- `GET /api/progress/:userId` — Retrieves historical progression entries.

2. Time-Series Audit Log Schema

```
{
  "_id": ObjectId("..."),
  "userId": ObjectId("6acf8a79dd378122a8942df1"),
  "timestamp": ISODate("2026-10-14T14:20:00.000Z"),
  "action": "SKILL_LEVEL_UP",
  "skillName": "MongoDB",
  "oldLevel": 2,
  "newLevel": 4,
  "notes": "Achieved level 4 in MongoDB"
}
```

3. Dual Operation on Level-Up

1. **State Mutation**: Performs an atomic `$set` on `users` to update current proficiency.
2. **Event Recording**: Inserts an append-only snapshot into `progressLogs`. This separation preserves historical telemetry without bloating the primary profile document.

Module 9: Interactive Web Dashboard & Reporting Module

1. Purpose & Responsibilities

The user interface delivering live data visualization, interactive modals, and print report generation. Built as a lightweight, zero-build Single Page Application (SPA).

- **Primary Files**: `public/index.html`
- **Client Technologies**: HTML5, Modern CSS (Tailwind CSS CDN), Vanilla JavaScript (ES6+ Fetch API), Chart.js CDN, FontAwesome.

2. Key UI Components

1. **Dynamic Metric Cards**: Real-time display of Career Readiness Score %, Missing Skills, Weak Skills, and Total Priority Gap Points.
2. **Chart.js Dual-Bar Visualization**:
3. Green bars = Candidate's Current Level ($0 \to 5$).
4. Indigo bars = Required Industry Benchmark ($1 \to 5$).
5. **Skill Manager Modal**: Dedicated interface to add any of the 39 CS skills, adjust sliders, and delete competencies.
6. **Printable Skill Audit Report (`@media print`)**: Formats candidate metadata, target role details, gap analysis matrix, and recommended courses into a clean, printable PDF lab sheet.

Module Summary Matrix

#	Module Name	Primary File	Primary MongoDB Feature
1	User Profile & Skills	`src/routes/users.ts`	Hybrid Embedding Pattern & Atomic `$set`
2	Career Taxonomy	`src/routes/targetRoles.ts`	Normalized Reference Pattern & BSON Storage
3	Core Aggregation Engine	`src/routes/skillGap.ts`	10-Stage Aggregation Pipeline (`$lookup` , `$unwind` , `$filter` , `$max`)
4	Multi-Role Fit & Readiness	`src/routes/skillGap.ts`	Weighted Mathematical Scoring & Cross-Role Evaluation
5	Learning Recommendations	`src/data/recommendations.json`	Foreign Field Indexing & Join via `$lookup`
6	Resume Skill Extractor	`src/routes/skills.ts`	Compound Text Index & Regex Token Matching
7	"What-If" Simulator	`src/routes/skillGap.ts`	In-Memory Scenario Modeling
8	Progress Audit Tracking	`src/routes/progress.ts`	Append-Only Audit Logging & Snapshot Pattern
9	Web UI & Reporting	`public/index.html`	Client-Side State Hydration & Chart.js Visualization